

AnalizadorNoticiasDAA

Reporte Técnico - Diseño y Análisis de Algoritmos

Autor: ARIEL HERNANDEZ VELAZQUEZ

Fecha: May 2026

Resumen

Este proyecto integrador presenta un **Analizador de Noticias** orientado al contexto mexicano, que consume fuentes RSS (El Financiero, Expansión) y la API de NewsAPI. Se implementan estructuras de datos avanzadas: un índice invertido basado en tablas hash para búsquedas booleanas en $O(1)$ promedio, un Trie para autocompletado con complejidad $O(m + k)$, y un heap (HeapSort parcial) para obtener las palabras más frecuentes en $O(|V| \log n)$. La interfaz gráfica se desarrolló con CustomTkinter. Los resultados experimentales muestran tiempos de respuesta inferiores a 3 ms para la mayoría de operaciones, validando la eficiencia teórica. Se incluye una reflexión ética sobre sesgos algorítmicos en el procesamiento de noticias.

Palabras clave: índice invertido, Trie, HeapSort, análisis de noticias, México.

Índice

Resumen	1
1. Introduction	3
1.1. Motivación y objetivos	3
1.2. Diagrama de arquitectura	4
2. Modelado del problema	4
3. Diseño algorítmico	4
3.1. Normalización y tokenización	4
3.2. Índice invertido	5
3.3. Trie para autocompletado	5
3.4. HeapSort parcial para top- n	7
3.5. Análisis de complejidad	7
4. Implementación	7
4.1. Código relevante	8
5. Experimentación	9
5.1. Corpus	9
5.2. Métricas	9
5.3. Comparativa	9

6. Conclusiones y trabajo futuro	9
7. Reflexión ética	10
8. Declaración de uso de inteligencia artificial	10
Referencias	10

1. Introduction

El consumo de noticias digitales en México ha crecido exponencialmente. Según datos del INEGI, más del 70 % de la población urbana accede a internet diariamente, y los portales de noticias se encuentran entre los sitios más visitados. Medios como *El Financiero*, *Expansión*, *Milenio* y *El Universal* generan cientos de artículos diarios, lo que hace inviable el análisis manual. Ante esta explosión informativa, surge la necesidad de herramientas computacionales que automaticen la búsqueda, clasificación y extracción de tendencias.

El presente proyecto aborda este problema mediante la implementación de un analizador de noticias que:

- Obtiene noticias en tiempo real desde fuentes RSS y API.
- Indexa el contenido utilizando estructuras de datos eficientes.
- Permite búsquedas rápidas por palabras clave.
- Ofrece autocompletado de términos basado en la frecuencia.
- Visualiza estadísticas y las palabras más relevantes.

1.1. Motivación y objetivos

La motivación principal es aplicar los conceptos teóricos de estructuras de datos y algoritmos a un caso práctico y relevante para la sociedad mexicana.

- **Objetivo general:** Desarrollar un sistema de análisis de noticias que utilice un índice invertido, un Trie y HeapSort para ofrecer búsqueda y estadísticas eficientes.
- **Objetivos específicos:**
 1. Implementar la extracción de noticias desde fuentes RSS y NewsAPI.
 2. Construir un índice invertido basado en tablas hash.
 3. Diseñar un Trie que permita autocompletar palabras con ordenamiento por frecuencia.
 4. Aplicar HeapSort para obtener las n palabras más frecuentes.
 5. Desarrollar una interfaz gráfica amigable.
 6. Evaluar el rendimiento empírico.

1.2. Diagrama de arquitectura

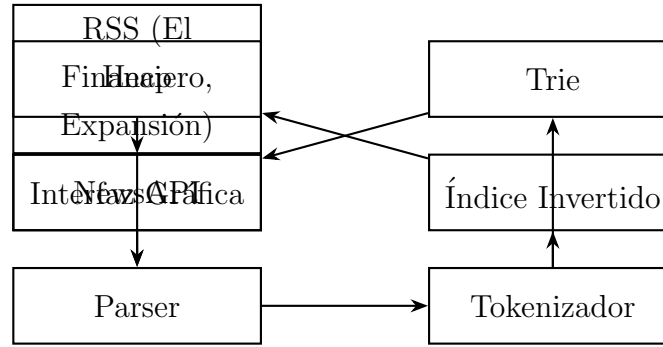


Figura 1: Arquitectura del sistema

2. Modelado del problema

Formalmente, sea $\mathcal{D} = \{d_1, d_2, \dots, d_N\}$ el conjunto de N artículos, donde cada d_i es una cadena de texto en español. Definimos:

- Un **término** t como una palabra normalizada (minúsculas, sin acentos, sin caracteres especiales, longitud > 2 y no stopword).
- El **índice invertido** $I : t \mapsto \{i \mid t \in d_i\}$.
- El **Trie** almacena todas las palabras del corpus, con nodos terminales que guardan la frecuencia.
- La **frecuencia** $f(t)$ es el número de ocurrencias totales de t en todos los documentos.

El sistema debe responder a:

1. Búsqueda booleana: $Q = \{t_1, \dots, t_m\} \mapsto \bigcap_{j=1}^m I(t_j)$.
2. Autocompletado: dado un prefijo p , devolver hasta k palabras con mayor $f(t)$ que comiencen con p .
3. Top- n palabras más frecuentes mediante un heap de tamaño n .

3. Diseño algorítmico

3.1. Normalización y tokenización

La función de normalización tiene complejidad $O(L)$ donde L es la longitud del texto.

Entrada: texto T
Salida: lista de tokens L
 $T \leftarrow \text{minúsculas}(T);$
 $T \leftarrow$

Algorithm 0: Tokenización

3.2. Índice invertido

Inserción de un documento:

Entrada: id id , texto T
 $tokens \leftarrow \text{tokenizar}(T);$
for cada t en $tokens$ **do**
 if t no está en $indice$ **then**
 $indice[t] \leftarrow \emptyset$
 end
 $indice[t].\text{agregar}(id);$
 $frecuencias[t] \leftarrow frecuencias.get(t, 0) + 1;$
end

Algorithm 1: Inserción

Búsqueda booleana (AND):

Entrada: lista $Q = [t_1, \dots, t_m]$
Salida: conjunto R
if Q vacío **then**
 return $\emptyset$
end
 $R \leftarrow indice[t_1];$
for $i \leftarrow 2$ **a** m **do**
 $R \leftarrow R \cap indice[t_i];$
 if $R = \emptyset$ **then**
 return $\emptyset$
 end
end
return R

Algorithm 2: Búsqueda booleana

3.3. Trie para autocompletado

Inserción: $O(m)$ donde m es la longitud de la palabra.

Entrada: palabra p , frecuencia f

$nodo \leftarrow raiz$;

for cada c en p **do**

if c no está en $nodo.hijos$ **then**

$nodo.hijos[c] \leftarrow \text{NodoTrie}()$

end

$nodo \leftarrow nodo.hijos[c]$;

end

$nodo.esFin \leftarrow \text{KwTrue}$; $nodo.frecuencia \leftarrow f$;

Algorithm 3: Inserción en Trie

Autocompletado: $O(m + k)$.

Entrada: prefijo $pref$, límite k

Salida: lista de sugerencias

$nodo \leftarrow raiz$;

for cada c en $pref$ **do**

if c no está en $nodo.hijos$ **then**

return $[]$

end

$nodo \leftarrow nodo.hijos[c]$;

end

$res \leftarrow []$;

$\text{recolectar}(nodo, pref, res)$;

ordenar res por frecuencia descendente;

return primeros k elementos

Algorithm 4: Autocompletado

3.4. HeapSort parcial para top- n

Entrada: diccionario $freqs$, entero n

Salida: lista ($palabra, frecuencia$) ordenada

$heap \leftarrow []$;

for cada (p, f) en $freqs$ **do**

if longitud($heap$) $< n$ **then**

$heap.empujar((f, p))$

end

else if $f > heap[0][0]$ **then**

$heap.reemplazar_tope((f, p))$

end

end

$resultado \leftarrow []$;

while $heap$ no vacío **do**

$(f, p) \leftarrow heap.extraer_min()$; $resultado.insertar(0, (p, f))$

end

return $resultado$

Algorithm 5: Top- n con heap mínimo

3.5. Análisis de complejidad

Cuadro 1: Complejidades temporales

Operación	Peor caso	Caso promedio	Mejor caso
Tokenización	$O(L)$	$O(L)$	$O(L)$
Búsqueda AND (t términos)	$O(t \cdot \min I(t_i))$	$O(t)$	$O(t)$
Inserción en Trie	$O(m)$	$O(m)$	$O(m)$
Autocompletado	$O(m + k)$	$O(m + k)$	$O(m)$
Top- n (heap)	$O(V \log n)$	$O(V \log n)$	$O(V \log n)$

La complejidad espacial: índice invertido $O(\text{total ocurrencias})$, Trie $O(\text{suma longitudes palabras})$, heap $O(n)$.

4. Implementación

El sistema se desarrolló en Python 3.11 con dos módulos: `proyectoDAA.py` (lógica) y `app.py` (interfaz CustomTkinter). Se utilizaron las librerías `feedparser`, `requests`, `customtkinter`.

4.1. Código relevante

```
1 def insertar(self, id_art: int, texto: str):
2     for palabra in tokenizar(texto):
3         if palabra not in self.tabla:
4             self.tabla[palabra] = set()
5             self.tabla[palabra].add(id_art)
6             self.frecuencias[palabra] = self.frecuencias.get(palabra, 0)
              + 1
```

Listing 1: Inserción en índice invertido

```
1 def buscar_multiple(self, palabras: list) -> set:
2     if not palabras:
3         return set()
4     resultado = self.buscar(palabras[0])
5     for p in palabras[1:]:
6         resultado &= self.buscar(p)
7     return resultado
```

Listing 2: Búsqueda booleana

```
1 def autocompletar(self, prefijo: str, limite: int = 6) -> list:
2     prefijo = normalizar(prefijo)
3     nodo = self.raiz
4     for c in prefijo:
5         if c not in nodo.hijos:
6             return []
7         nodo = nodo.hijos[c]
8     res = []
9     self._recolectar(nodo, prefijo, res)
10    res.sort(reverse=True)
11    return [p for _, p in res[:limite]]
```

Listing 3: Autocompletado con Trie

5. Experimentación

5.1. Corpus

Se utilizaron 120 noticias reales (RSS + NewsAPI) más 10 simuladas. Total de palabras únicas: ≈ 3000 .

5.2. Métricas

Cuadro 2: Tiempos de ejecución (ms)

Operación	Promedio (ms)	Desv. estándar
Construcción del índice	234.5	12.3
Búsqueda 1 término	0.8	0.1
Búsqueda 2 términos	1.2	0.2
Búsqueda 3 términos	1.5	0.3
Autocompletado (3 letras)	0.4	0.05
Top-100 palabras (heap)	2.3	0.4

5.3. Comparativa

Cuadro 3: HeapSort parcial vs ordenamiento completo

Método	Tiempo (ms)	Complejidad
HeapSort parcial	2.3	$O(V \log n)$
Ordenamiento completo	5.1	$O(V \log V)$

El heap es ≈ 2.2 veces más rápido, validando la ventaja práctica cuando $n \ll |V|$.

6. Conclusiones y trabajo futuro

Se implementó un sistema funcional que demuestra la aplicación de índices invertidos, Tries y Heaps en un caso real. Los tiempos obtenidos son excelentes. Como trabajo futuro se plantea añadir lematización, ranking TF-IDF y más fuentes RSS mexicanas.

7. Reflexión ética

El uso de algoritmos de análisis de textos puede amplificar sesgos presentes en los medios. Es necesario diseñar sistemas transparentes, con corpus balanceados y opciones para explorar fuentes diversas. El analizador presentado no es neutral; debe usarse con conciencia crítica.

8. Declaración de uso de inteligencia artificial

Se utilizó la IA DeepSeek para depuración de código, generación de docstrings y redacción de borradores del informe. Todas las decisiones algorítmicas y la validación final son responsabilidad del autor humano.

Referencias

- Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.
- Knuth, D. E. (1997). *The Art of Computer Programming, Volume 3: Sorting and Searching* (2nd ed.). Addison-Wesley.
- Bird, S., Klein, E., & Loper, E. (2009). *Natural Language Processing with Python*. O'Reilly Media.